

LAPORAN KELOMPOK 2
SISTEM KASIR BERBASIS DEKSTOP

Dosen : Saiful Nur Budiman, S.Kom., M.Kom



Disusun Oleh:

Rendra Adnan Farid	25104410036
M. Rizki Desca F.	25104410041
Ema Netasari	25104410052
Nahrul Mubarak	25104410062
Gladis Ayu Diapitaloka	25104410073
Adelia Wahyu Bima Sakti	25104410094

UNIVERSITAS ISLAM BALITAR
PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI DAN INFORMASI
TAHUN 2026

PENDAHULUAN

Aplikasi Manajemen Toko adalah sistem desktop berbasis Java Swing untuk mengelola data produk dan kategori. Aplikasi ini menggunakan database MySQL dan menerapkan arsitektur MVC (Model-View-Controller) serta pola DAO (Data Access Object).

1. CARA MENJALANKAN APLIKASI

Langkah-langkah:

A. Persiapan Database

1. Buka phpMyAdmin atau MySQL Workbench
2. Buat database baru bernama `db\_toko`
3. Import file `db\_toko.sql` yang disediakan

B. Menjalankan Aplikasi Melalui IDE (NetBeans/Eclipse)

1. Buka project di IDE
2. Pastikan library MySQL Connector sudah ditambahkan
3. Jalankan file `FormProduk.java` sebagai main class

C. Tampilan Awal Aplikasi

Penjelasan Tampilan:

- **Header:** Menampilkan judul "List Produk" dan tombol "Kategori" untuk membuka form manajemen kategori
- **Tabel:** Menampilkan data produk dengan kolom Kode, Nama Produk, Harga, Stock, Kategori, dan Brand
- **Tombol Aksi:** Load (refresh data), Insert (tambah produk baru), Update (edit produk), Delete (hapus produk)

2. PENGGUNAAN INHERITANCE

A. DialogInputProduk → java.awt.Dialog

```
public class DialogInputProduk extends java.awt.Dialog {
```

DialogInputProduk` mewarisi class `java.awt.Dialog` yang merupakan komponen untuk membuat jendela dialog di Java.

(Properties dari Dialog) mewarisi:

1. Kemampuan untuk muncul sebagai window modal

2. Event handling bawaan (windowClosing)
3. Method setVisible(), dispose(), pack(), setLocationRelativeTo()

Manfaat Inheritance pada DialogInputProduk:

- Tidak perlu membuat window dari nol
- Mendapatkan fitur window seperti border, title bar, tombol close
- Dapat menggunakan layout manager untuk mengatur komponen
- Kompatibel dengan event handling system Java

B. FormKategori → javax.swing.JFrame

```
public class FormKategori extends javax.swing.JFrame {
```

FormKategori` mewarisi class `javax.swing.JFrame` yang merupakan komponen utama untuk membuat jendela aplikasi di Swing.

(properties dari JFrame) mewarisi:

1. Window Utama Aplikasi
2. Method setSize(width, height);, setPreferredSize(new Dimension(width, height));, setVisible(true/false);, getContentPane().setLayout(new BorderLayout());
3. Container untuk Komponen
4. Event Handling

Manfaat Inheritance pada FormKategori:

- Mendapatkan semua fungsionalitas window standar
- Dapat dijalankan sebagai aplikasi independen
- Mendukung berbagai layout manager
- Terintegrasi dengan Swing component hierarchy
- Dapat menggunakan method seperti pack(), setVisible(), dispose()

C. FormProduk → javax.swing.JFrame

```
public class FormProduk extends javax.swing.JFrame {
```

FormProduk juga mewarisi javax.swing.JFrame dengan fungsi yang sama seperti FormKategori, namun dengan penambahan fitur spesifik untuk manajemen produk.

Kelebihan Inheritance dari JFrame pada FormProduk:

1. Dapat Membuka Dialog Lain
2. Dapat Menjadi Parent untuk Dialog
3. Fungsi sebagai Aplikasi Utama

D. Konsep Inheritance yang Diterapkan

Superclass → Subclass Relationship:



E. Keuntungan Inheritance pada Aplikasi

1. Code Reusability (Penggunaan Ulang Kode)

- Tidak perlu menulis ulang kode untuk membuat window
- Menggunakan method standar seperti setVisible(), dispose(), setTitle()

2. Konsistensi Antarmuka

- Semua form memiliki title bar, tombol close, border yang sama
- User experience yang konsisten

3. Fungsi yang Sudah Siap Pakai

- Event handling (window events, key events, mouse events)
- Layout management
- Focus management

4. Mudah Dipelihara

- Perubahan pada superclass akan mempengaruhi semua subclass
- Misal: menambah logging saat window dibuka

3. POLA MVC DAN DAO

A. Arsitektur MVC (Model-View-Controller)

Pengertian MVC:

MVC adalah pola arsitektur yang memisahkan aplikasi menjadi 3 komponen utama:

Model: Menangani data dan logika bisnis

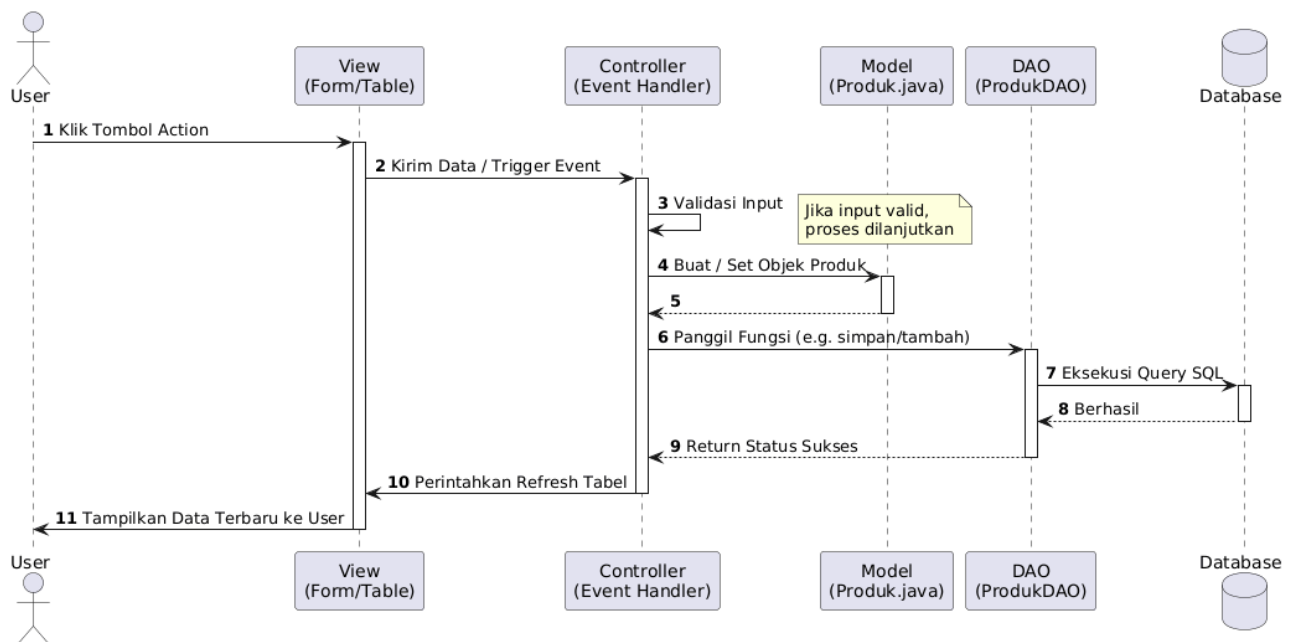
View: Menangani tampilan/user interface

Controller: Menangani logika aplikasi dan menghubungkan Model dengan View

Implementasi di Aplikasi:

Komponen	Package	Contoh Class	Fungsi Spesifik
Model	ClassModel	Produk.java Kategori.java	- Menyimpan atribut data - Getter dan Setter - Representasi tabel database
View	GUI	FormProduk.java FormKategori.java DialogInputProduk.java	- Menampilkan data ke user - Menerima input dari user - Komponen Swing (JTable, JButton, dll)
Controller	GUI + Dao	Event handler di Form ProdukDAO KategoriDAO	- Memproses input user - Validasi data - Mengupdate Model dan View - Akses database

Alur Data dalam MVC:



Contoh Implementasi MVC:

Model (Produk.java):

```
public class Produk {  
    // ID unik produk  
    private int kode;  
  
    // ID kategori (foreign key) - buat nyambungin ke tabel kategori  
    private int kodeProduk;  
  
    // Nama produk, contoh: "TV 32 Inch" atau "Sepatu Lari"  
    private String nama;  
  
    // Harga jual (dalam Rupiah)  
    private int harga;  
  
    // Jumlah stok barang di gudang  
    private int stock;  
  
    /**  
     * Atribut tambahan (gak ada di database).  
     * Ini hasil JOIN dari tabel produk dan kategori.  
     * Biar pas ditampilkan di JTable langsung keliatan nama kategori dan brand-nya,  
     * tanpa perlu query lagi.  
     */  
    private String namaKategori; // Nama kategori, contoh: "Elektronik"  
    private String brand;        // Brand, contoh: "Samsung"
```

View + Controller (FormProduk.java):

```
public class FormProduk extends javax.swing.JFrame {  
    // Objek DAO buat akses database produk  
    private Dao.ProdukDAO produkDao = new Dao.ProdukDAO();  
  
    private void btninsertActionPerformed(java.awt.event.ActionEvent evt) {  
        // Panggil form dialog  
        GUI.DialogInputProduk dialog = new GUI.DialogInputProduk(this, true, produkDao);  
        dialog.setVisible(true);  
    }  
  
    void loadDataToTable() {  
        // Ambil model tabelnya  
        javax.swing.table.DefaultTableModel model = (javax.swing.table.DefaultTableModel) tablelistproduk.getModel();  
  
        // Kosongkan isi tabel dulu  
        model.setRowCount(0);  
  
        // Ambil semua data produk dari database (lengkap dengan JOIN kategori)  
        this.listProduk = produkDao.getAllProduk();  
  
        // Masukkan satu per satu ke tabel  
        for (ClassModel.Produk p : listProduk) {  
            Object[] row = {  
                p.getKode(),  
                p.getNama(),  
                p.getHarga(),  
                p.getStock(),  
                p.getNamaKategori(), // Hasil JOIN dari tabel kategori  
                p.getBrand()         // Hasil JOIN dari tabel kategori  
            };  
            model.addRow(row);  
        }  
    }  
}
```

DAO (ProdukDAO.java):

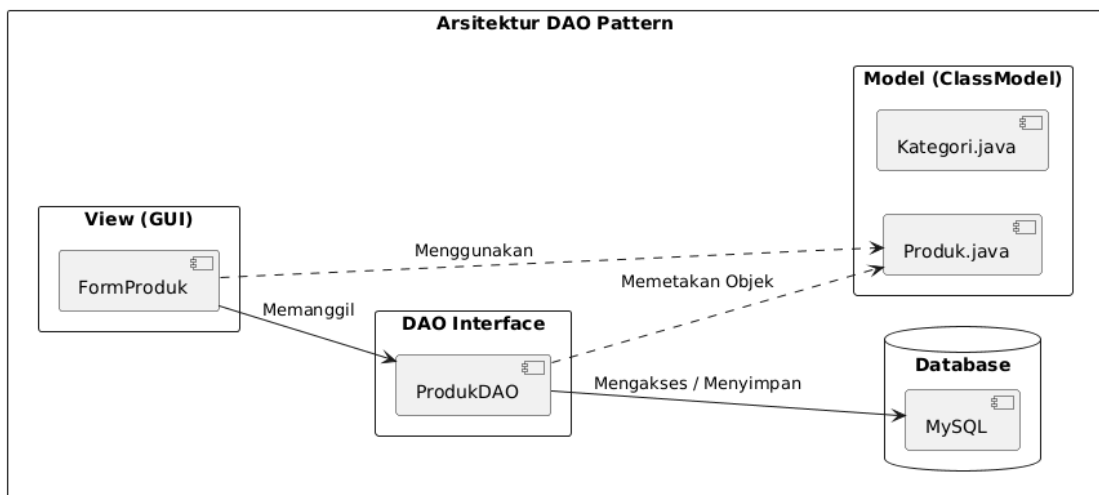
```
public class ProdukDAO {  
    // Controller: Mengelola akses database  
    public List<Produk> getAllProduk() { ... }  
    public boolean insert(Produk p) { ... }  
    public boolean update(Produk p) { ... }  
    public boolean delete(int kode) { ... }  
}
```

B. DAO (Data Access Object)

Pengertian DAO:

DAO adalah pola desain yang digunakan untuk mengabstraksi dan mengkapsulasi semua akses ke database. DAO menyediakan interface untuk operasi CRUD (Create, Read, Update, Delete) tanpa mengekspos detail implementasi database.

Struktur DAO:



1. KategoriDAO - Detail Implementasi:

```
public class KategoriDAO {
    private Connection conn;

    public KategoriDAO() {
        this.conn = DBKoneksi.getConnection(); // Ambil koneksi database
    }

    // CREATE: Insert data baru
    public boolean insert(Kategori k) {
        String query = "INSERT INTO kategori (kategori, brand) VALUES (?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(query)) {
            ps.setString(1, k.getKategori());
            ps.setString(2, k.getBrand());
            return ps.executeUpdate() > 0; // Return true jika berhasil
        }
    }

    // READ: Ambil semua data
    public List<Kategori> getAllKategori() {
        List<Kategori> list = new ArrayList<>();
        String query = "SELECT * FROM kategori ORDER BY kode DESC";
        // ... eksekusi query dan return list
        return list;
    }

    // UPDATE: Ubah data
    public boolean update(Kategori k) {
        String query = "UPDATE kategori SET kategori = ?, brand = ? WHERE kode = ?";
        // ... eksekusi update
    }

    // DELETE: Hapus data
    public boolean delete(int kode) {
        String query = "DELETE FROM kategori WHERE kode = ?";
        // ... eksekusi delete
    }

    // Fungsi tambahan: Cek relasi foreign key
    public boolean isDipakaiProduk(int kodeKategori) {
        String query = "SELECT COUNT(*) FROM produk WHERE kode_produk = ?";
        // ... return true jika ada produk yang menggunakan
    }
}
```


2. ProdukDAO - Detail Implementasi:

```
public class ProdukDAO {
    private Connection conn;

    public ProdukDAO() {
        this.conn = DBKoneksi.getConnection();
    }

    // CREATE: Insert produk baru
    public boolean insert(Produk p) {
        String query = "INSERT INTO produk (kode_produk, nama, harga, stock) VALUES (?, ?, ?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(query)) {
            ps.setInt(1, p.getKodeProduk()); // Foreign key ke kategori
            ps.setString(2, p.getNama());
            ps.setInt(3, p.getHarga());
            ps.setInt(4, p.getStock());
            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
            return false;
        }
    }
}
```

```
// READ: Ambil semua produk dengan JOIN
public List<Produk> getAllProduk() {
    List<Produk> list = new ArrayList<>();
    String query = "SELECT p.*, k.kategori, k.brand " +
        "FROM produk p " +
        "INNER JOIN kategori k ON p.kode_produk = k.kode " +
        "ORDER BY p.kode DESC";

    // ... eksekusi query
    // Set data produk dan tambahan dari JOIN
    p.setNamaKategori(rs.getString("kategori"));
    p.setBrand(rs.getString("brand"));
    return list;
}

// READ: Ambil produk by ID
public Produk getProdukById(int kode) {
    // ... query dengan WHERE kode = ?
}

// UPDATE: Ubah produk
public boolean update(Produk p) {
    // ... UPDATE produk WHERE kode = ?
}

// DELETE: Hapus produk
public boolean delete(int kode) {
    // ... DELETE FROM produk WHERE kode = ?
}
}
```

C. DBKoneksi - Singleton Pattern

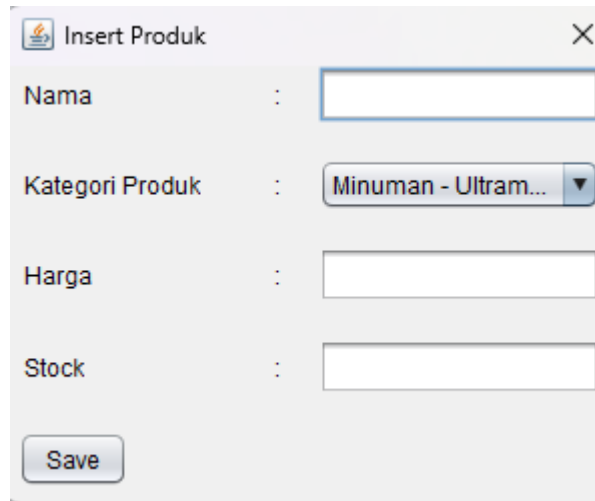
```
public class DBKoneksi {  
    // Setting koneksi database  
    private static final String URL = "jdbc:mysql://localhost:3306/db_toko";  
    private static final String USER = "root";  
    private static final String PASSWORD = "";  
  
    private static Connection connection = null;  
  
    /**  
     * Method untuk mendapatkan koneksi database.  
     * Koneksi cuma dibuat sekali (pattern Singleton),  
     * kalau udah ada bakal dipake lagi.  
     *  
     * @return objek Connection buat konek ke database  
     */  
    public static Connection getConnection() {  
        if (connection == null) {  
            try {  
                // Load driver MySQL  
                Class.forName("com.mysql.cj.jdbc.Driver");  
  
                // Bikin koneksi ke database  
                connection = DriverManager.getConnection(URL, USER, PASSWORD);  
                System.out.println("Koneksi ke database db_toko BERHASIL!");  
            } catch (ClassNotFoundException e) {  
                System.err.println("Driver MySQL tidak ketemu: " + e.getMessage());  
            } catch (SQLException e) {  
                System.err.println("Koneksi database GAGAL: " + e.getMessage());  
            }  
        }  
        return connection;  
    }  
}
```

Keuntungan Singleton Pattern:

- Koneksi database dibuat hanya sekali
- Menghemat resource (tidak buat koneksi baru setiap kali)
- Konsisten untuk semua DAO

D. Alur CRUD dengan DAO

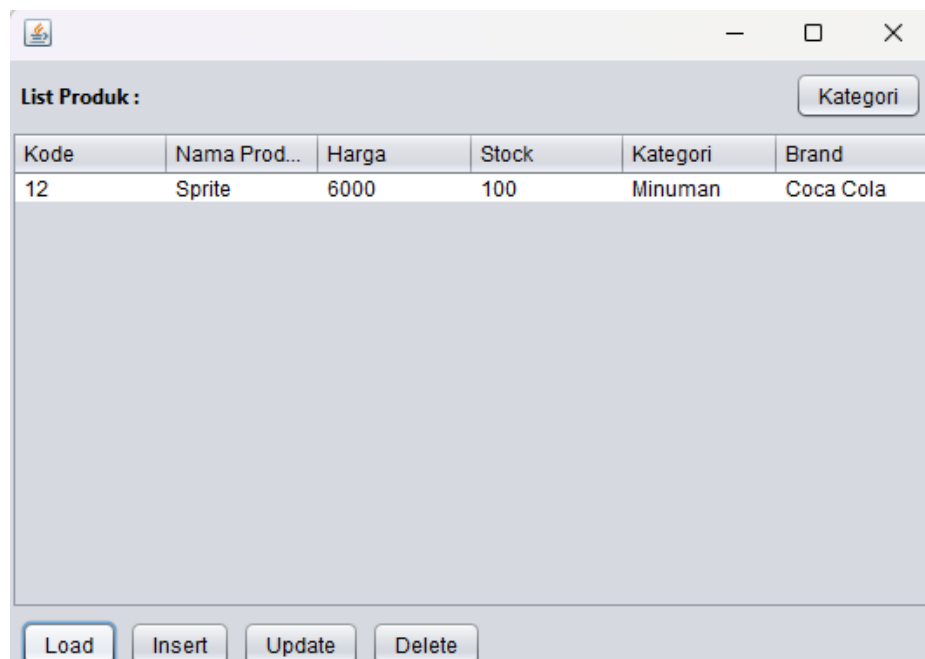
CREATE (Insert):



User Klik "Insert"

1. FormProduk buka DialogInputProduk
2. User isi data dan klik "Save"
3. DialogInputProduk panggil produkDao.insert(p)
4. ProdukDAO eksekusi INSERT SQL
5. Jika berhasil, FormProduk.loadDataToTable() refresh tampilan

READ (Load):

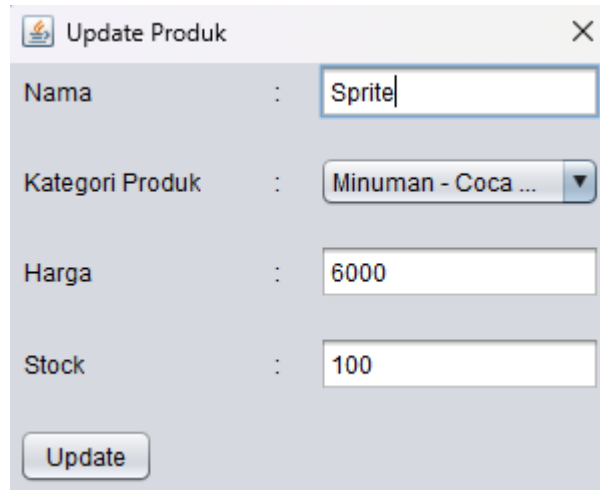


Kode	Nama Prod...	Harga	Stock	Kategori	Brand
12	Sprite	6000	100	Minuman	Coca Cola

User Klik "Load"

1. FormProduk panggil loadDataToTable()
2. loadDataToTable panggil produkDao.getAllProduk()
3. ProdukDAO eksekusi SELECT dengan JOIN
4. Data ditampilkan di Jtable

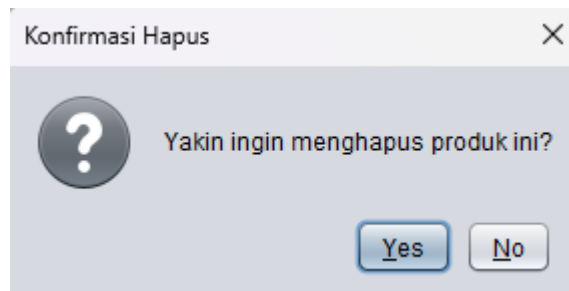
UPDATE:



User pilih produk dan klik "Update"

1. FormProduk ambil data produk dari tabel
2. Tampilkan dialog update dengan data lama
3. User ubah data dan klik "Update"
4. Dialog panggil produkDao.update(p)
5. ProdukDAO eksekusi UPDATE SQL
6. Jika berhasil, refresh tabel

DELETE:



User pilih produk dan klik "Delete"

1. Konfirmasi penghapusan
2. FormProduk panggil produkDao.delete(kode)

3. ProdukDAO eksekusi DELETE SQL
4. Jika berhasil, refresh tabel

4. PENANGANAN ERROR FOREIGN KEY

A. Pengertian Foreign Key

Foreign Key adalah kolom yang merujuk ke PRIMARY KEY di tabel lain.

Dalam aplikasi ini:



B. Cek Ketergantungan Sebelum Hapus

Masalah: Jika kategori memiliki produk terkait, menghapus kategori akan menyebabkan error karena:

1. Data produk menjadi invalid (referensi ke kategori yang tidak ada)
2. Melanggar aturan foreign key constraint

Solusi: Cek terlebih dahulu apakah kategori masih dipakai oleh produk

1. Method isDipakaiProduk di KategoriDAO:

```

/**
 * Cek apakah kategori ini masih dipakai oleh produk.
 * Biar gak bisa dihapus kalau masih ada produk yang pake.
 *
 * @param kodeKategori ID kategori yang mau dicek
 * @return true kalau masih dipakai, false kalau aman dihapus
 */
public boolean isDipakaiProduk(int kodeKategori) {
    String query = "SELECT COUNT(*) FROM produk WHERE kode_produk = ?";

    try (PreparedStatement ps = conn.prepareStatement(query)) {
        ps.setInt(1, kodeKategori);

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getInt(1) > 0;
            }
        }
    } catch (SQLException e) {
        System.err.println("Error cek relasi produk: " + e.getMessage());
    }

    return false;
}
  
```

Penjelasan Query:

- SELECT COUNT(*) → Menghitung jumlah produk
- WHERE kode_produk = ? → Mencari produk dengan kategori tertentu
- Jika count > 0 → Kategori masih dipakai

2. Implementasi di FormKategori - btndeleteActionPerformed:

```
private void btndeleteActionPerformed(java.awt.event.ActionEvent evt) {
    // Cek apakah ada baris yang dipilih
    int selectedRow = tblkategori.getSelectedRow();
    if (selectedRow == -1) {
        javax.swing.JOptionPane.showMessageDialog(this, "Pilih kategori yang akan dihapus.");
        return;
    }

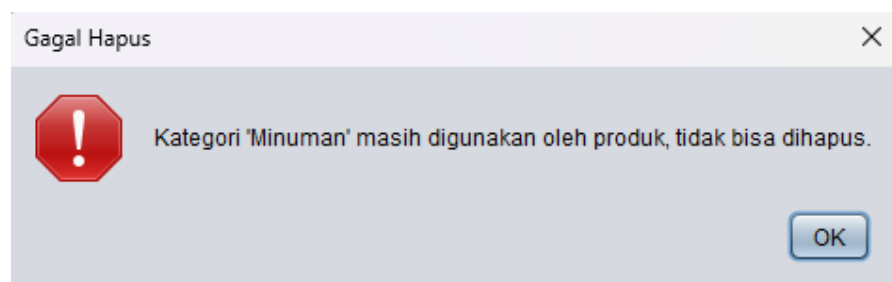
    // Ambil data dari baris yang dipilih
    int kode = (int) tblkategori.getValueAt(selectedRow, 0);
    String namaKategori = (String) tblkategori.getValueAt(selectedRow, 1);

    // Cek apakah kategori masih dipakai oleh produk
    if (kategoriDao.isDipakaiProduk(kode)) {
        javax.swing.JOptionPane.showMessageDialog(
            this,
            "Kategori '" + namaKategori + "' masih digunakan oleh produk, tidak bisa dihapus.",
            "Gagal Hapus",
            javax.swing.JOptionPane.ERROR_MESSAGE
        );
        return;
    }

    // Konfirmasi penghapusan
    int confirm = javax.swing.JOptionPane.showConfirmDialog(
        this,
        "Yakin ingin menghapus kategori '" + namaKategori + " '?",
        "Konfirmasi Hapus",
        javax.swing.JOptionPane.YES_NO_OPTION
    );

    if (confirm == javax.swing.JOptionPane.YES_OPTION) {
        if (kategoriDao.delete(kode)) {
            javax.swing.JOptionPane.showMessageDialog(this, "Kategori berhasil dihapus.");
            loadKategoriToTable(); // Refresh tabel
        } else {
            javax.swing.JOptionPane.showMessageDialog(this, "Gagal menghapus kategori.");
        }
    }
}
```

Tampilan Error:



C. Penanganan Error Database Lainnya

1. SQLException Handling:

Semua operasi database dibungkus dengan try-catch untuk menangani error.

2. Error Duplikat Data (Unique Constraint):

Database memiliki constraint UNIQUE pada kolom nama produk:

```
ALTER TABLE produk ADD UNIQUE KEY `nama` (`nama`);
```

Penanganan melalui validasi sebelum insert/update:

```
// Cek apakah kombinasi kategori + brand sudah ada di database
if (kategoriDao.isDuplikat(namaKategori, brand)) {
    javax.swing.JOptionPane.showMessageDialog(
        this,
        "Gagal! Kategori '" + namaKategori + "' dengan Brand '" + brand + "' sudah ada.",
        "Data Duplikat",
        javax.swing.JOptionPane.ERROR_MESSAGE
    );
    return;
}
```

3. Error Koneksi Database:

```
try {
    // Load driver MySQL
    Class.forName("com.mysql.cj.jdbc.Driver");

    // Bikin koneksi ke database
    connection = DriverManager.getConnection(URL, USER, PASSWORD);
    System.out.println("Koneksi ke database db_toko BERHASIL!");
} catch (ClassNotFoundException e) {
    System.err.println("Driver MySQL tidak ketemu: " + e.getMessage());
} catch (SQLException e) {
    System.err.println("Koneksi database GAGAL: " + e.getMessage());
}
```

5. VALIDASI PADA KODE

Pengertian Validasi

Validasi adalah proses memeriksa apakah input atau data memenuhi kriteria tertentu sebelum diproses lebih lanjut. Tujuan validasi:

- Mencegah data tidak valid masuk ke sistem
- Memberikan feedback yang jelas kepada user
- Menjaga integritas data
- Mencegah error runtime

A. Validasi Field Kosong

Lokasi: `DialogInputProduk.java` - method `btnsaveActionPerformed()`

Penjelasan: Validasi ini memastikan bahwa semua field input telah diisi oleh user sebelum data disimpan. Jika ada field yang kosong, proses penyimpanan akan dihentikan dan user akan diberikan notifikasi.

Kode Implementasi:

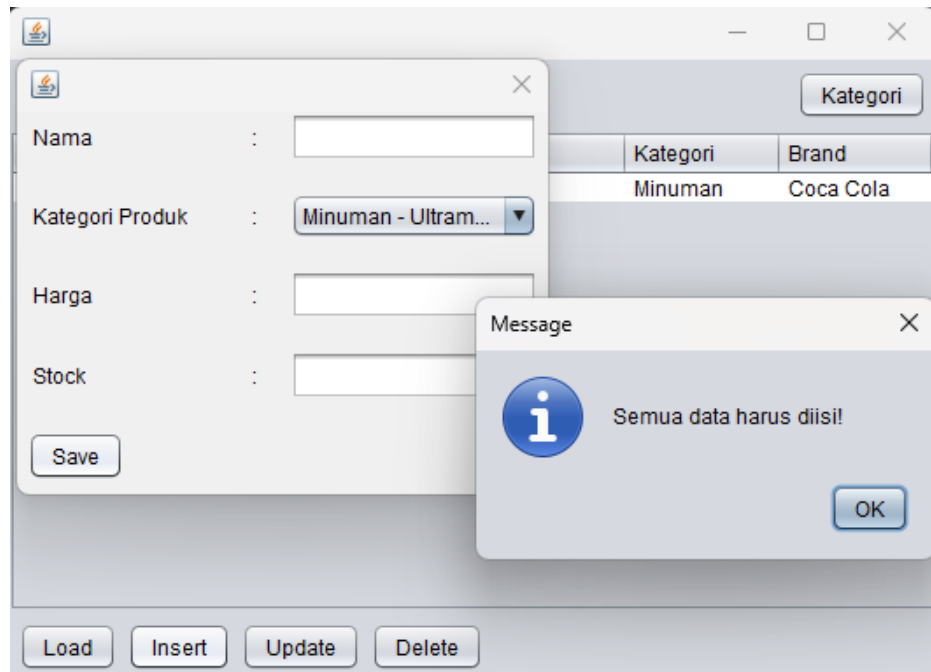
```
/**
 * Tombol Save: simpan produk baru ke database.
 * Validasi yang dilakukan:
 * 1. Cek field kosong
 * 2. Cek tipe data angka (NumberFormatException)
 * 3. Cek nilai negatif
 */
private void btnsaveActionPerformed(java.awt.event.ActionEvent evt) {
    // Ambil semua input dari form
    String nama = txtnama.getText().trim();
    int kategoriIndex = cmbkategoriproduk.getSelectedIndex();
    String hargaStr = txtharga.getText().trim();
    String stokStr = txtstock.getText().trim();

    // VALIDASI 1: Cek apakah ada data yang kosong
    if (nama.isEmpty() || kategoriIndex == -1 || hargaStr.isEmpty() || stokStr.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Semua data harus diisi!");
        return;
    }
}
```

Penjelasan Kode:

- **txtnama.getText().trim():** Mengambil teks dari field nama dan menghilangkan spasi di awal/akhir
- **isEmpty():** Mengecek apakah string kosong (panjang = 0)
- **kategoriIndex == -1:** Mengecek apakah user belum memilih kategori di combobox
- **JOptionPane.showMessageDialog():** Menampilkan dialog pesan error ke user
- **return:** Menghentikan eksekusi method sehingga data tidak disimpan

Tampilan Error:



B. Validasi Tipe Data Angka

Lokasi: `DialogInputProduk.java` - method `btnsaveActionPerformed()`
dan `FormProduk.java` - method `showUpdateDialog()`

Penjelasan: Validasi ini memastikan bahwa input untuk harga dan stock adalah angka yang valid. Java akan melempar `NumberFormatException` jika string tidak bisa dikonversi ke integer.

Kode Implementasi:

```
try {
    // VALIDASI 2: Konversi harga dan stok ke angka
    int harga = Integer.parseInt(hargaStr);
    int stok = Integer.parseInt(stokStr);

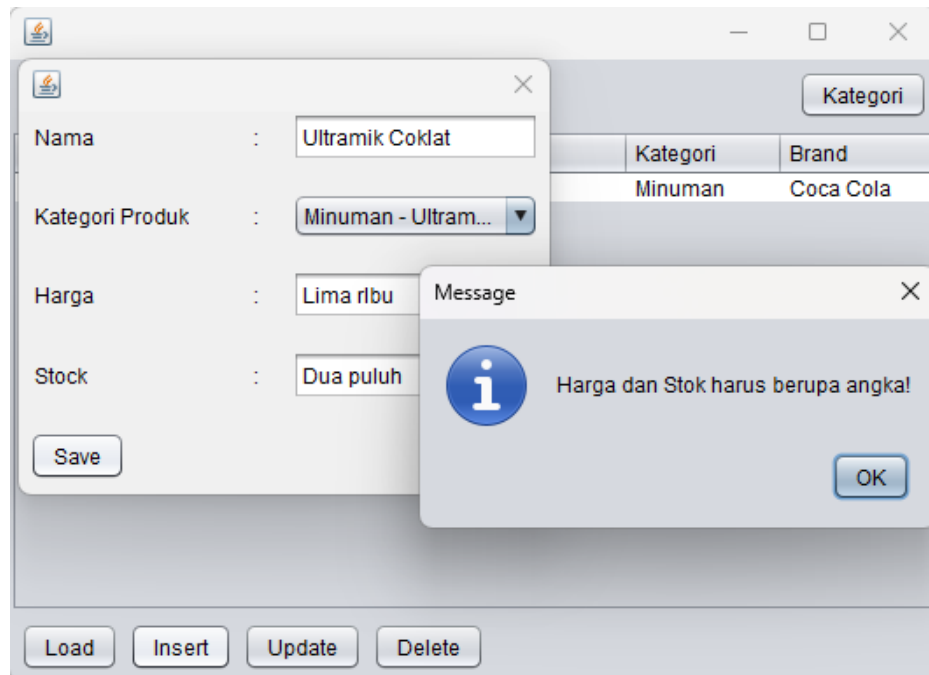
} catch (NumberFormatException ex) {
    // Kalau harga atau stok bukan angka
    javax.swing.JOptionPane.showMessageDialog(this, "Harga dan Stok harus berupa angka!");
}
```

Penjelasan Kode:

- **Integer.parseInt():** Method untuk mengkonversi String menjadi int
- **NumberFormatException:** Exception yang dilempar jika string tidak bisa dikonversi
- **try-catch:** Menangkap exception agar aplikasi tidak crash

- Pesan error yang jelas memberitahu user apa yang salah

Tampilan Error:



C. Validasi Nilai Negatif

Lokasi: 'FormProduk.java - method showUpdateDialog()

Penjelasan: Validasi ini memastikan bahwa harga dan stock tidak boleh bernilai negatif. Harga dan stock barang tidak mungkin negatif dalam konteks bisnis.

Kode Implementasi 'DialogInputProduk':

```
try {
    // VALIDASI 2: Konversi harga dan stok ke angka
    int harga = Integer.parseInt(hargaStr);
    int stok = Integer.parseInt(stokStr);

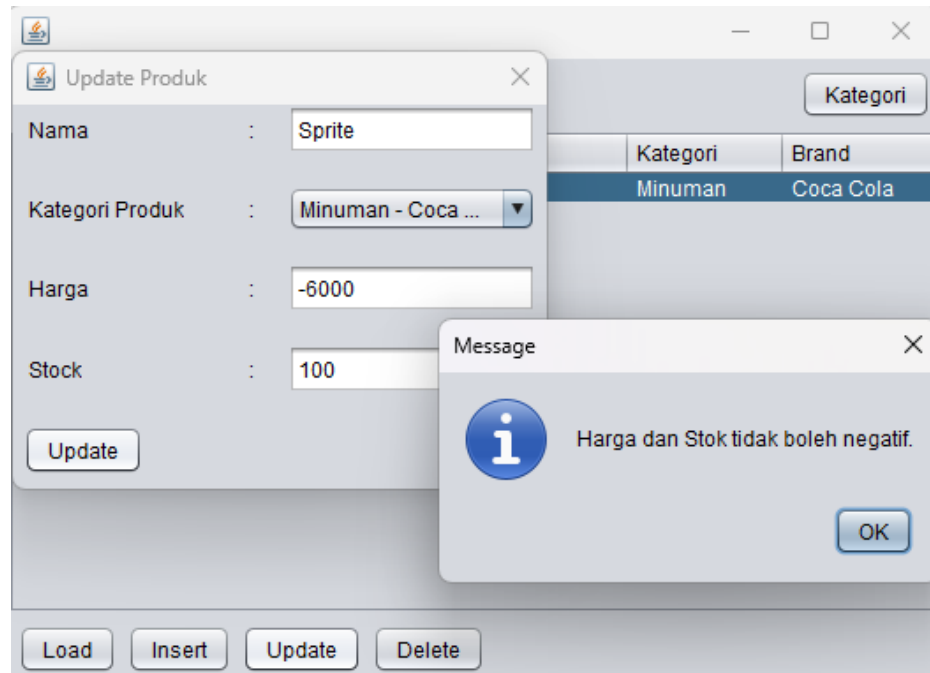
    // VALIDASI 3: Cek apakah nilai negatif
    if (harga < 0 || stok < 0) {
        javax.swing.JOptionPane.showMessageDialog(this,
            "Harga dan Stok tidak boleh negatif!");
        return;
    }
}
```

Penjelasan Kode:

- **harga < 0:** Mengecek apakah harga kurang dari 0
- **stok < 0:** Mengecek apakah stock kurang dari 0

- Kondisi menggunakan OR ('||'): Jika salah satu negatif, error ditampilkan

Tampilan Error:



D. Validasi Duplikat Insert

Lokasi: FormKategori.java - method btninsertActionPerformed()

Penjelasan: Validasi ini mencegah user menambahkan kategori dengan kombinasi nama kategori dan brand yang sudah ada di database. Ini menjaga keunikan data.

Kode Implementasi `FormKategori.java` :

```

private void btninsertActionPerformed(java.awt.event.ActionEvent evt) {
    // Minta input nama kategori
    String namaKategori = javax.swing.JOptionPane.showInputDialog(
        this,
        "Masukkan Nama Kategori:",
        "Input Kategori",
        javax.swing.JOptionPane.PLAIN_MESSAGE
    );
    if (namaKategori == null) return; // User klik cancel

    // Validasi: nama kategori gak boleh kosong
    namaKategori = namaKategori.trim();
    if (namaKategori.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Nama Kategori tidak boleh kosong!");
        return;
    }

    // Minta input brand
    String brand = javax.swing.JOptionPane.showInputDialog(
        this,
        "Masukkan Nama Brand:",
        "Input Brand",
        javax.swing.JOptionPane.PLAIN_MESSAGE
    );
    if (brand == null) return; // User klik cancel

    // Validasi: brand gak boleh kosong
    brand = brand.trim();
    if (brand.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Brand tidak boleh kosong!");
        return;
    }

    // Cek apakah kombinasi kategori + brand sudah ada di database
    if (kategoriDao.isDuplikat(namaKategori, brand)) {
        javax.swing.JOptionPane.showMessageDialog(
            this,
            "Gagal! Kategori '" + namaKategori + "' dengan Brand '" + brand + "' sudah ada.",
            "Data Duplikat",
            javax.swing.JOptionPane.ERROR_MESSAGE
        );
        return;
    }
}

```

Method isDuplikat di KategoriDAO:

```

public boolean isDuplikat(String kategori, String brand) {
    String query = "SELECT COUNT(*) FROM kategori WHERE kategori = ? AND brand = ?";

    try (PreparedStatement ps = conn.prepareStatement(query)) {
        ps.setString(1, kategori);
        ps.setString(2, brand);

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getInt(1) > 0;
            }
        }
    } catch (SQLException e) {
        System.err.println("Error cek duplikat: " + e.getMessage());
    }
    return false;
}

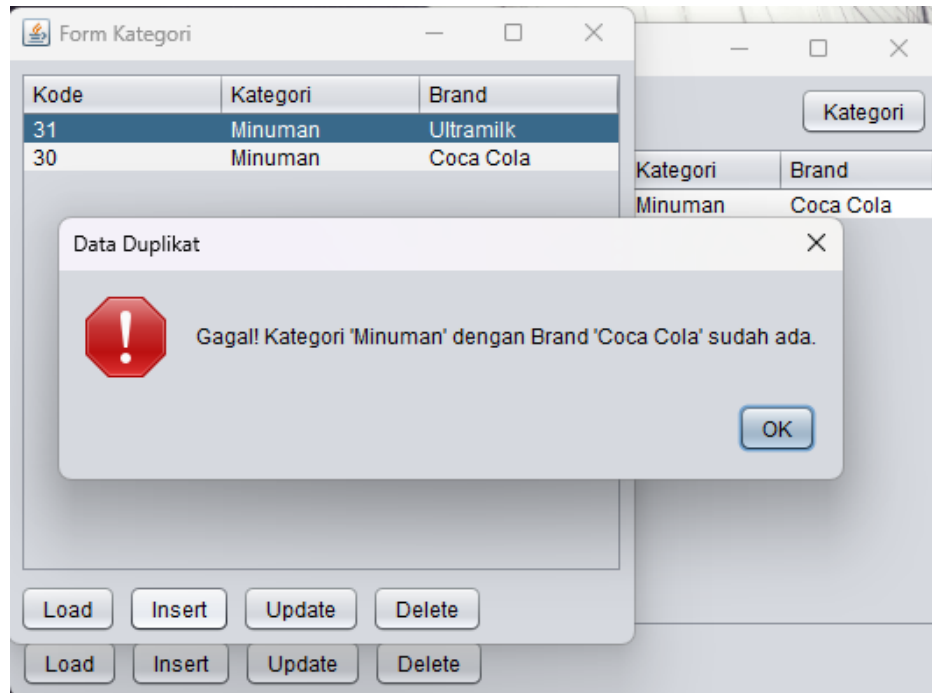
```

Penjelasan Kode:

- isDuplikat(): Method untuk mengecek duplikat di database
- Query COUNT: Menghitung jumlah data dengan kombinasi yang sama

- Jika count > 0: Berarti data sudah ada → tampilkan error
- JOptionPane.ERROR_MESSAGE: Menampilkan icon error pada dialog

Tampilan Error:



E. Validasi Duplikat Update

Lokasi: `FormKategori.java - method btnupdateActionPerformed()

Penjelasan: Validasi ini mirip dengan validasi duplikat insert, namun dengan pengecualian: data yang sama persis dengan data lama tidak dianggap duplikat (karena user tidak mengubah apapun).

Kode Implementasi FormKategori.java` :

```

private void btnupdateActionPerformed(java.awt.event.ActionEvent evt) {
    // Cek apakah ada baris yang dipilih
    int selectedRow = tblkategori.getSelectedRow();
    if (selectedRow == -1) {
        javax.swing.JOptionPane.showMessageDialog(this, "Pilih kategori yang akan diupdate.");
        return;
    }

    // Ambil data lama dari baris yang dipilih
    int kode = (int) tblkategori.getValueAt(selectedRow, 0);
    String kategoriLama = (String) tblkategori.getValueAt(selectedRow, 1);
    String brandLama = (String) tblkategori.getValueAt(selectedRow, 2);

    // Minta input kategori baru (dengan default data lama)
    String kategoriBaru = javax.swing.JOptionPane.showInputDialog(
        this,
        "Masukkan Nama Kategori baru:",
        kategoriLama
    );
    if (kategoriBaru == null) return; // User klik cancel
    kategoriBaru = kategoriBaru.trim();
    if (kategoriBaru.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Nama Kategori tidak boleh kosong.");
        return;
    }

    // Minta input brand baru (dengan default data lama)
    String brandBaru = javax.swing.JOptionPane.showInputDialog(
        this,
        "Masukkan Brand baru:",
        brandLama
    );
    if (brandBaru == null) return; // User klik cancel
    brandBaru = brandBaru.trim();
    if (brandBaru.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Brand tidak boleh kosong.");
        return;
    }

    // Cek duplikat (kecuali datanya sama persis dengan yang lama)
    if (kategoriDao.isDuplikat(kategoriBaru, brandBaru)) {
        if (!(kategoriLama.equals(kategoriBaru) && brandLama.equals(brandBaru))) {
            javax.swing.JOptionPane.showMessageDialog(
                this,
                "Kombinasi Kategori dan Brand sudah ada!",
                "Duplikat",
                javax.swing.JOptionPane.ERROR_MESSAGE
            );
            return;
        }
    }

    // Bungkus data ke objek Kategori
    Kategori k = new Kategori(kode, kategoriBaru, brandBaru);

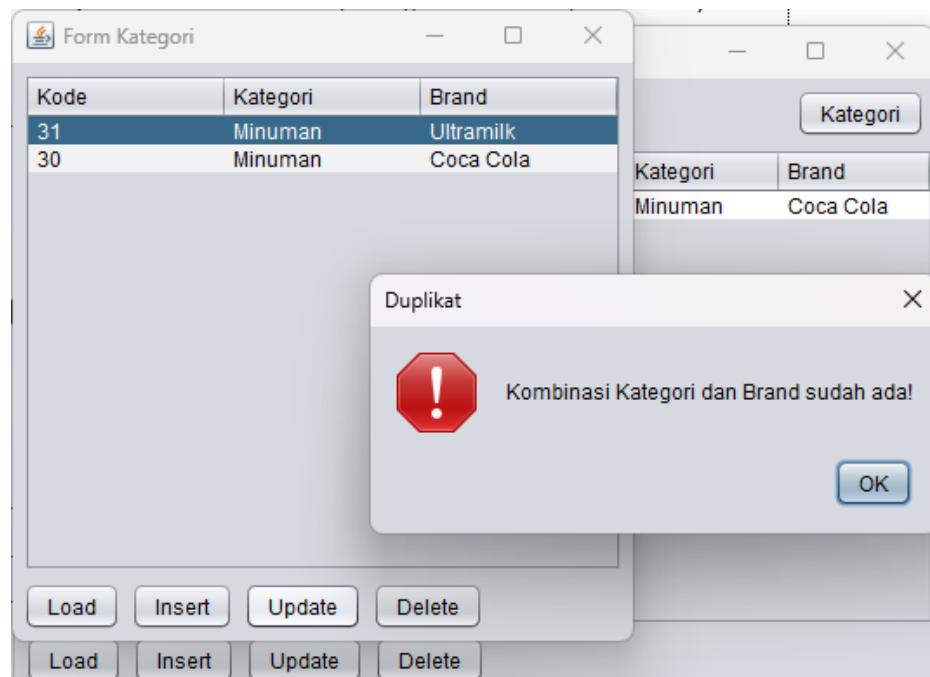
    // Simpan perubahan ke database
    if (kategoriDao.update(k)) {
        javax.swing.JOptionPane.showMessageDialog(this, "Kategori berhasil diupdate.");
        loadKategoriToTable(); // Refresh tabel
    } else {
        javax.swing.JOptionPane.showMessageDialog(this, "Gagal mengupdate kategori.");
    }
}

```

Penjelasan Kode:

- ``kategoriLama.equals(kategoriBaru)``: Membandingkan kategori lama dengan baru
- ``brandLama.equals(brandBaru)``: Membandingkan brand lama dengan baru
- **Logika:** Jika data baru sama dengan data lama, abaikan error duplikat
- memungkinkan user mengklik Update tanpa mengubah apapun

Tampilan Validasi:



F. Validasi Seleksi Delete

Lokasi: ``FormKategori.java`` - method ``btndeleteActionPerformed()`` dan ``FormProduk.java`` - method ``btndeleteActionPerformed()``

Penjelasan: Validasi ini memastikan bahwa user telah memilih baris di tabel sebelum melakukan operasi delete. Ini mencegah error ketika user mengklik Delete tanpa memilih data.

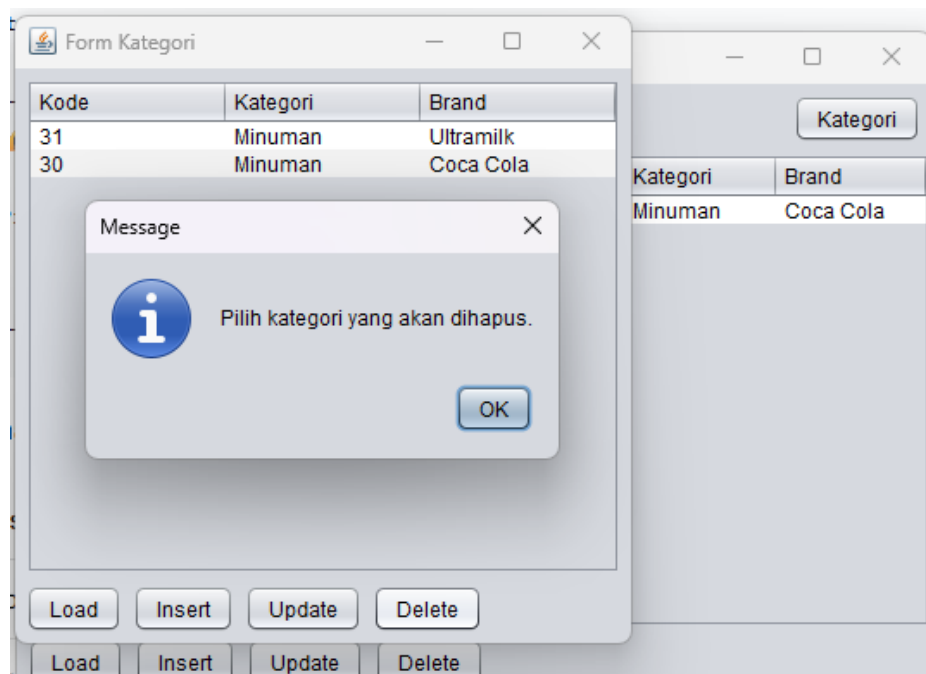
Kode Implementasi:

```
private void btndeleteActionPerformed(java.awt.event.ActionEvent evt) {  
    // Cek apakah ada baris yang dipilih  
    int selectedRow = tblkategori.getSelectedRow();  
    if (selectedRow == -1) {  
        javax.swing.JOptionPane.showMessageDialog(this, "Pilih kategori yang akan dihapus.");  
        return;  
    }  
}
```

Penjelasan Kode:

- **getSelectedRow():** Mengembalikan index baris yang dipilih, atau -1 jika tidak ada
- **selectedRow == -1:** Kondisi ketika tidak ada baris yang dipilih
- Pesan error memberitahu user untuk memilih data terlebih dahulu
- **return:** Menghentikan proses delete

Tampilan Validasi:



G. Validasi Seleksi Update

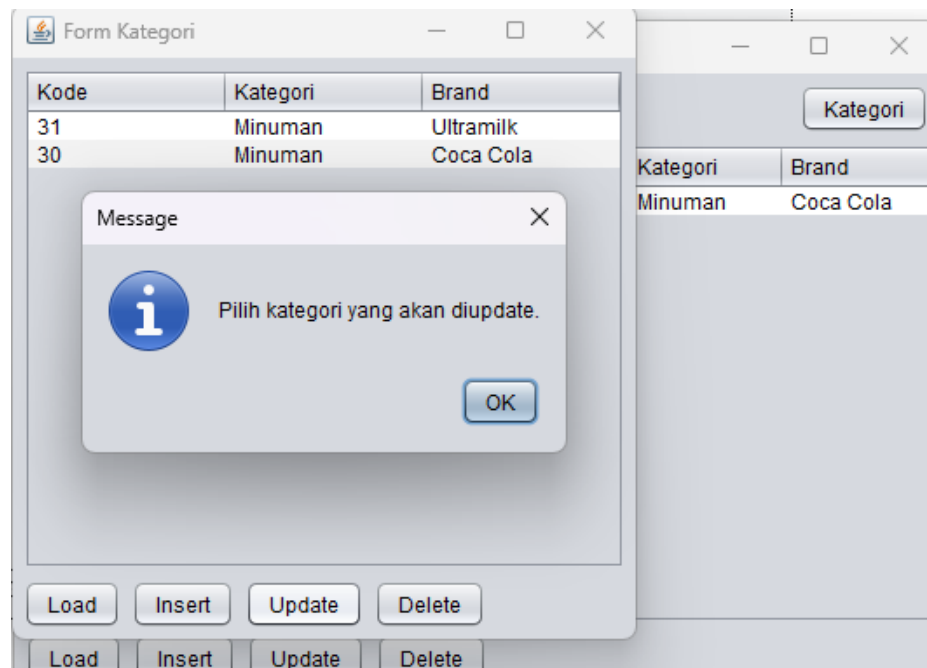
Lokasi: `FormKategori.java` - method `btndeleteActionPerformed()` dan
`FormProduk.java` - method `btnupdateActionPerformed()`

Penjelasan: Validasi ini memastikan bahwa user telah memilih baris di tabel sebelum melakukan operasi update. Sama seperti validasi delete, ini mencegah error.

Kode Implementasi:

```
private void btnupdateActionPerformed(java.awt.event.ActionEvent evt) {  
    // Cek apakah ada baris yang dipilih  
    int selectedRow = tblkategori.getSelectedRow();  
    if (selectedRow == -1) {  
        javax.swing.JOptionPane.showMessageDialog(this, "Pilih kategori yang akan diupdate.");  
        return;  
    }  
}
```

Tampilan Validasi:



H. Validasi Konfirmasi Hapus

Lokasi: `FormKategori.java` - method `btndeleteActionPerformed()` dan
`FormProduk.java` - method `btndeleteActionPerformed()`

Penjelasan: Validasi ini memberikan kesempatan kepada user untuk mengkonfirmasi atau membatalkan operasi hapus. Ini mencegah penghapusan data yang tidak disengaja.

Kode Implementasi:

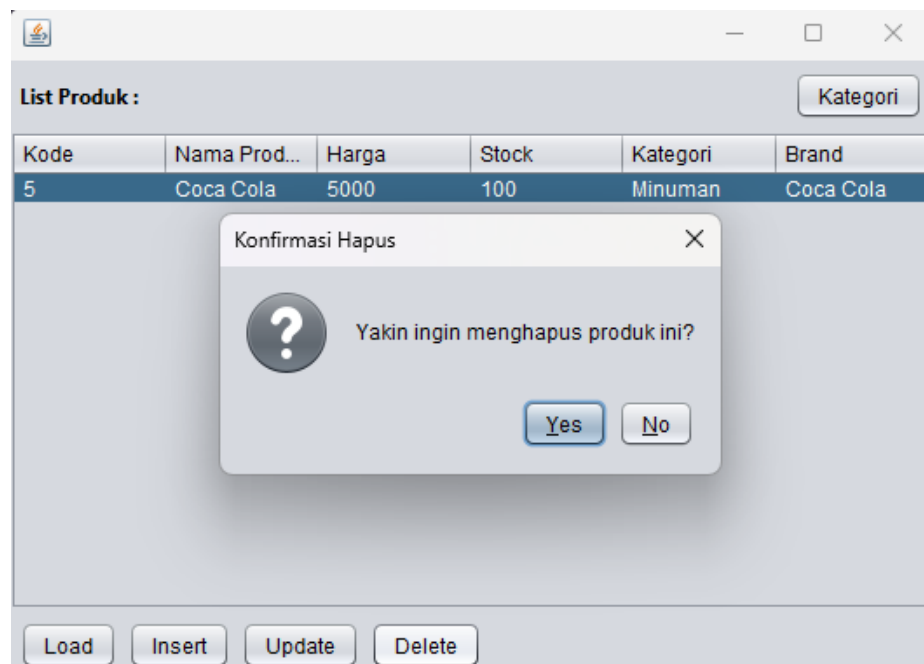
```
// Konfirmasi penghapusan
int confirm = javax.swing.JOptionPane.showConfirmDialog(
    this,
    "Yakin ingin menghapus kategori '" + namaKategori + "'",
    "Konfirmasi Hapus",
    javax.swing.JOptionPane.YES_NO_OPTION
);

if (confirm == javax.swing.JOptionPane.YES_OPTION) {
    if (kategoriDao.delete(kode)) {
        javax.swing.JOptionPane.showMessageDialog(this, "Kategori berhasil dihapus.");
        loadKategoriToTable(); // Refresh tabel
    } else {
        javax.swing.JOptionPane.showMessageDialog(this, "Gagal menghapus kategori.");
    }
}
```

Penjelasan Kode:

- `JOptionPane.showConfirmDialog()`: Menampilkan dialog dengan tombol Yes dan No
- `JOptionPane.YES_NO_OPTION`: Menentukan tombol yang tersedia (Yes dan No)
- `JOptionPane.YES_OPTION`: Konstanta untuk tombol Yes (nilai = 0)
- `JOptionPane.NO_OPTION`: Konstanta untuk tombol No (nilai = 1)
- Hanya jika user memilih Yes, operasi hapus dieksekusi

Tampilan Dialog Konfirmasi:



I. Validasi Data Tidak Ditemukan

Lokasi: `FormProduk.java` - method `btnupdateActionPerformed()`

Penjelasan: Validasi ini memastikan bahwa data produk yang akan diupdate benar-benar ada di database. Ini menangani skenario di mana data mungkin sudah dihapus oleh user lain atau terjadi inkonsistensi data.

Kode Implementasi `FormProduk.java`:

```
private void btnupdateActionPerformed(java.awt.event.ActionEvent evt) {  
    int selectedRow = tablelistproduk.getSelectedRow();  
    if (selectedRow == -1) {  
        JOptionPane.showMessageDialog(this, "Pilih produk yang akan diupdate.");  
        return;  
    }  
  
    int kodeProduk = (int) tablelistproduk.getValueAt(selectedRow, 0);  
  
    Produk p = produkDao.getProdukById(kodeProduk);  
    if (p == null) {  
        JOptionPane.showMessageDialog(this, "Data produk tidak ditemukan.");  
        return;  
    }  
  
    showUpdateDialog(p);  
}
```

Penjelasan Kode:

- ``produkDao.getProdukById()``: Mengambil data produk dari database berdasarkan kode
- Method ini mengembalikan null jika produk tidak ditemukan
- ``p == null``: Kondisi ketika produk tidak ada di database
- Pesan error memberitahu user bahwa data sudah tidak ada

J. Validasi Kategori Tidak Valid

Lokasi: `FormProduk.java` - method `showUpdateDialog()`

Penjelasan: Validasi ini memastikan bahwa kategori yang dipilih dari combobox adalah kategori yang valid dan ada di database. Ini mencegah penyimpanan data dengan foreign key yang tidak valid.

Kode Implementasi:

```
private void showUpdateDialog(Produk produkLama)  
...  
  
// === Event handler tombol Update ===  
btnupdate.addActionListener((ActionEvent e) -> {  
    String nama = txtnama.getText().trim();  
    String hargaStr = txtharga.getText().trim();  
    String stokStr = txtstock.getText().trim();  
    String selectedItem = (String) cmbkategoriproduk.getSelectedItem();
```

```
Kategori kategoriTerpilih = null;
for (Kategori kat : listKategori) {
    String comboText = kat.getKategori() + " - " + kat.getBrand();
    if (comboText.equals(selectedItem)) {
        kategoriTerpilih = kat;
        break;
    }
}

if (kategoriTerpilih == null) {
    JOptionPane.showMessageDialog(updateDialog, "Kategori tidak valid.");
    return;
}
```

```
// Set foreign key ke produk
produkLama.setKodeProduk(kategoriTerpilih.getKode());
// ... lanjut proses update
});
}
```

Penjelasan Kode:

- ``cmbkategoriproduk.getSelectedItemAt()``: Mengambil string yang dipilih di combobox
- Loop melalui ``listKategori`` untuk mencari kategori yang cocok
- ``comboText.equals(selectedItem)``: Membandingkan text di combobox dengan data kategori
- Jika tidak ditemukan, kategori tidak valid → tampilkan error

K. Validasi User Cancel Input

Lokasi: ``FormKategori.java`` - method ``btninsertActionPerformed()``

Penjelasan: Validasi ini menangani skenario ketika user mengklik tombol Cancel pada dialog input (`JOptionPane.showInputDialog`). User mungkin membatalkan operasi input, dan aplikasi harus merespons dengan baik.

Kode Implementasi:

```

private void btninsertActionPerformed(java.awt.event.ActionEvent evt) {
    // Minta input nama kategori
    String namaKategori = javax.swing.JOptionPane.showInputDialog(
        this,
        "Masukkan Nama Kategori:",
        "Input Kategori",
        javax.swing.JOptionPane.PLAIN_MESSAGE
    );
    if (namaKategori == null) return; // User klik cancel

    // Validasi: nama kategori gak boleh kosong
    namaKategori = namaKategori.trim();
    if (namaKategori.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Nama Kategori tidak boleh kosong!");
        return;
    }

    // Minta input brand
    String brand = javax.swing.JOptionPane.showInputDialog(
        this,
        "Masukkan Nama Brand:",
        "Input Brand",
        javax.swing.JOptionPane.PLAIN_MESSAGE
    );
    if (brand == null) return; // User klik cancel

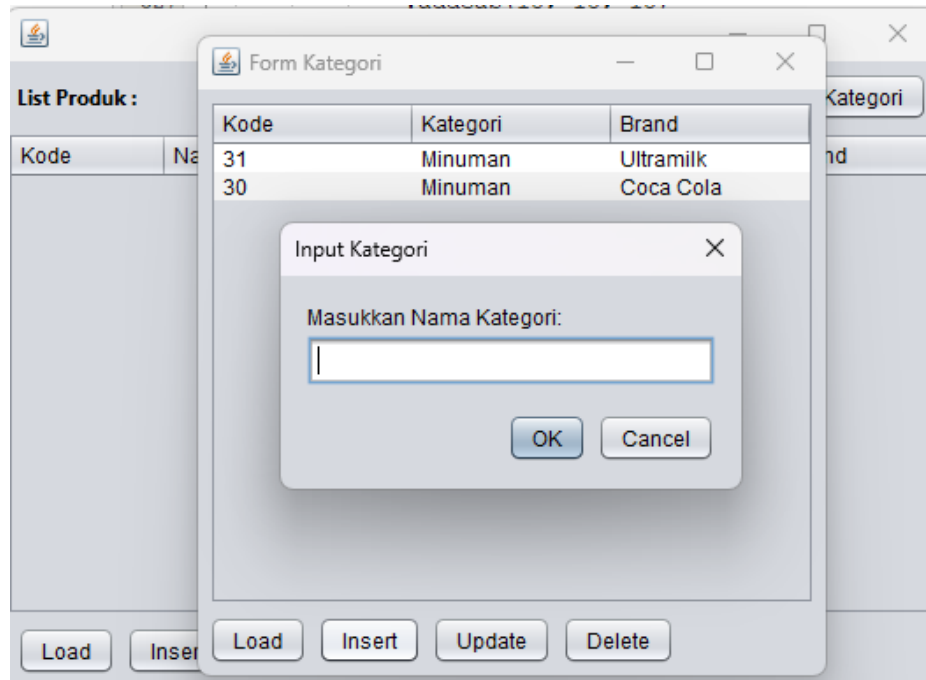
    // Validasi: brand gak boleh kosong
    brand = brand.trim();
    if (brand.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Brand tidak boleh kosong!");
        return;
    }
}

```

Penjelasan Kode:

- `JOptionPane.showInputDialog()`: Menampilkan dialog input dengan field text
- **Method ini mengembalikan:**
String yang diinput jika user klik OK
`null` jika user klik Cancel atau menutup dialog
- **Validasi `namaKategori == null`:** Mengecek apakah user membatalkan input
- Jika user cancel, method langsung `return` tanpa melanjutkan ke input berikutnya

Tampilan Dialog Input:



L. Dampak Validasi pada Aplikasi

1. Keamanan Data:

- Mencegah input tidak valid masuk ke database
- Menjaga integritas referensial (foreign key)
- Mencegah SQL injection (melalui prepared statement)

2. User Experience:

- Feedback yang jelas dan informatif
- Mencegah kebingungan user
- Mengurangi error yang tidak terduga

3. Stabilitas Aplikasi:

- Mencegah crash akibat NumberFormatException
- Menangani skenario edge case dengan baik
- Error handling yang terstruktur

4. Maintainability:

- Kode yang terstruktur dan mudah dipahami
- Validasi terkonsentrasi di satu tempat
- Mudah untuk menambah validasi baru

6. FITUR-FITUR KHUSUS

A. Auto Refresh Data

Pengertian: Auto refresh adalah mekanisme di mana data di tabel otomatis diperbarui (di-load ulang dari database) setelah operasi CRUD (Create, Read, Update, Delete) dilakukan.

Implementasi:

1. Method loadDataToTable di FormProduk:

```
void loadDataToTable() {
    // Ambil model tabelnya
    javax.swing.table.DefaultTableModel model = (javax.swing.table.DefaultTableModel) tablelistproduk.getModel();

    // Kosongkan isi tabel dulu
    model.setRowCount(0);

    // Ambil semua data produk dari database (lengkap dengan JOIN kategori)
    this.listProduk = produkDao.getAllProduk();

    // Masukkan satu per satu ke tabel
    for (ClassModel.Produk p : listProduk) {
        Object[] row = {
            p.getKode(),
            p.getNama(),
            p.getHarga(),
            p.getStock(),
            p.getNamaKategori(), // Hasil JOIN dari tabel kategori
            p.getBrand()         // Hasil JOIN dari tabel kategori
        };
        model.addRow(row);
    }
}
```

2. Auto Refresh Setelah Insert:

```
// Simpan ke database
if (produkDao.insert(p)) {
    javax.swing.JOptionPane.showMessageDialog(this, "Produk baru berhasil ditambahkan!");
    parentForm.loadDataToTable(); // Refresh tabel di form utama
    this.dispose(); // Tutup dialog
} else {
    javax.swing.JOptionPane.showMessageDialog(this, "Gagal menyimpan produk ke database.");
}
```

3. Auto Refresh Setelah Update:

```
if (produkDao.update(produkLama)) {
    JOptionPane.showMessageDialog(updateDialog, "Produk berhasil diupdate!");
    loadDataToTable();
    updateDialog.dispose();
}
```

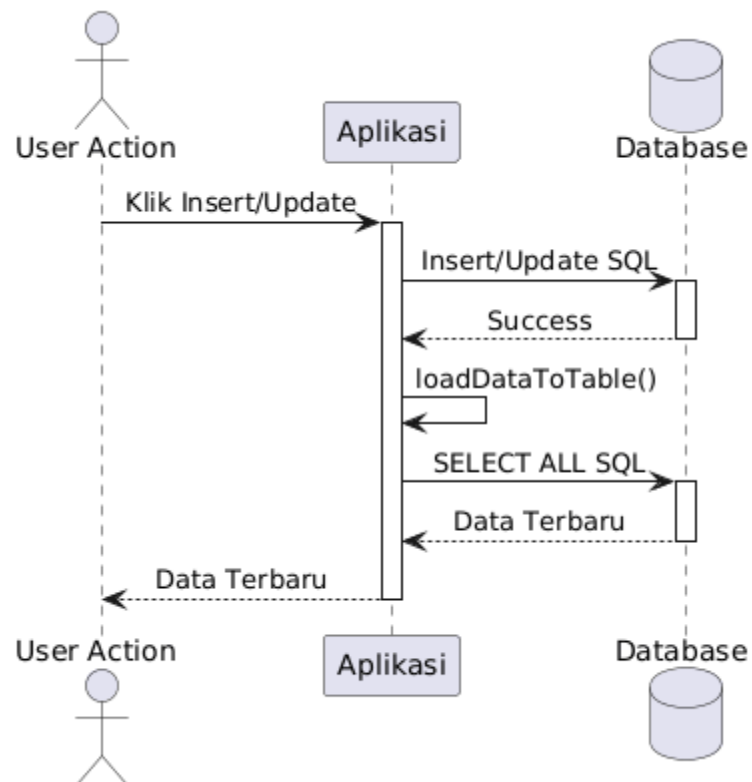
4. Auto Refresh Setelah Delete:

```
if (confirm == javax.swing.JOptionPane.YES_OPTION) {
    if (produkDao.delete(kodeProduk)) {
        javax.swing.JOptionPane.showMessageDialog(this, "Produk berhasil dihapus.");
        loadDataToTable(); // Refresh tabel
    }
}
```

5. Auto Refresh di FormKategori:

```
private void loadKategoriToTable() {  
    // Ambil model tabelnya  
    javax.swing.table.DefaultTableModel model = (javax.swing.table.DefaultTableModel) tblkategori.getModel();  
  
    // Kosongkan isi tabel dulu  
    model.setRowCount(0);  
  
    // Ambil semua data kategori dari database  
    java.util.List<Kategori> list = kategoriDao.getAllKategori();  
  
    // Masukkan satu per satu ke tabel  
    for (Kategori k : list) {  
        Object[] row = {  
            k.getKode(),  
            k.getKategori(),  
            k.getBrand()  
        };  
        model.addRow(row);  
    }  
}
```

Diagram Alur Auto Refresh:



B. Tidak Bisa Hapus Kategori yang Masih Dipakai Produk

Pengertian: Fitur ini mencegah pengguna menghapus kategori yang masih memiliki produk terkait. Ini menjaga integritas referensial data.

Fitur ini penting karena:

1. Data Integrity: Jika kategori dihapus, produk yang merujuk padanya menjadi invalid
2. Business Logic: Kategori yang sudah memiliki produk seharusnya tidak dihapus
3. Database Constraint: Foreign key constraint akan error jika dipaksakan

Implementasi:

1. Method isDipakaiProduk di KategoriDAO:

```
public boolean isDipakaiProduk(int kodeKategori) {  
    String query = "SELECT COUNT(*) FROM produk WHERE kode_produk = ?";  
  
    try (PreparedStatement ps = conn.prepareStatement(query)) {  
        ps.setInt(1, kodeKategori);  
  
        try (ResultSet rs = ps.executeQuery()) {  
            if (rs.next()) {  
                return rs.getInt(1) > 0;  
            }  
        }  
    } catch (SQLException e) {  
        System.err.println("Error cek relasi produk: " + e.getMessage());  
    }  
    return false;  
}
```

2. Penggunaan di FormKategori - Delete:

```

private void btdeleteActionPerformed(java.awt.event.ActionEvent evt) {
// Cek apakah ada baris yang dipilih
int selectedRow = tblkategori.getSelectedRow();
if (selectedRow == -1) {
    javax.swing.JOptionPane.showMessageDialog(this, "Pilih kategori yang akan dihapus.");
    return;
}

// Ambil data dari baris yang dipilih
int kode = (int) tblkategori.getValueAt(selectedRow, 0);
String namaKategori = (String) tblkategori.getValueAt(selectedRow, 1);

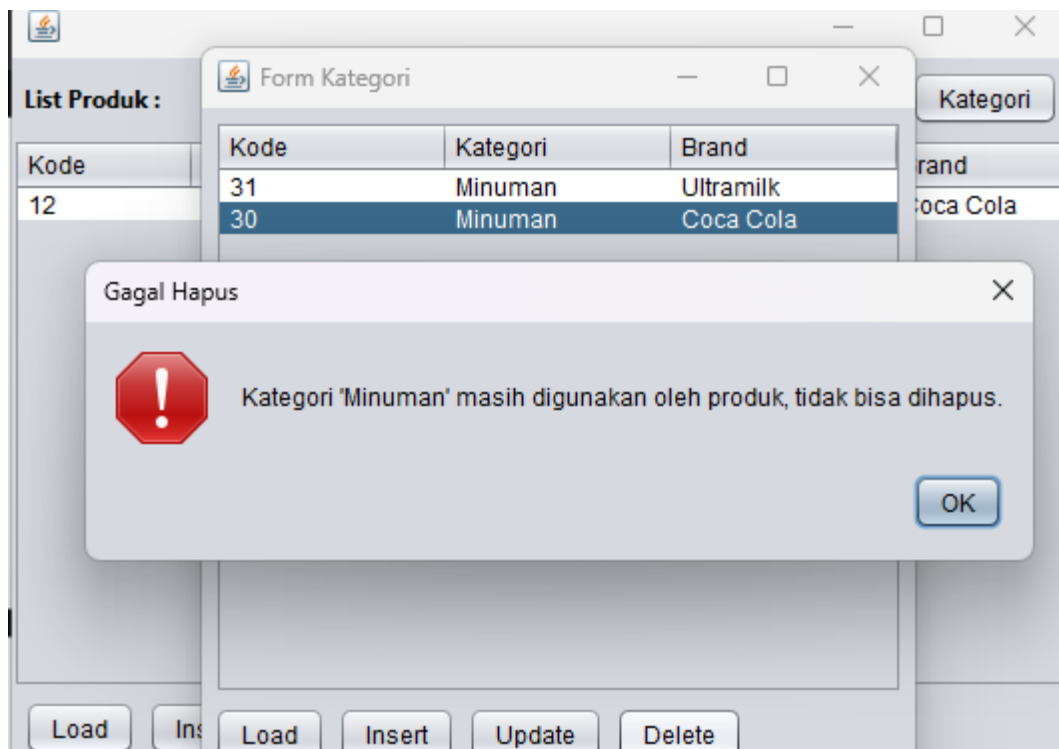
// Cek apakah kategori masih dipakai oleh produk
if (kategoriDao.isDipakaiProduk(kode)) {
    javax.swing.JOptionPane.showMessageDialog(
        this,
        "Kategori '" + namaKategori + "' masih digunakan oleh produk, tidak bisa dihapus.",
        "Gagal Hapus",
        javax.swing.JOptionPane.ERROR_MESSAGE
    );
    return;
}

// Konfirmasi penghapusan
int confirm = javax.swing.JOptionPane.showConfirmDialog(
    this,
    "Yakin ingin menghapus kategori '" + namaKategori + " '?",
    "Konfirmasi Hapus",
    javax.swing.JOptionPane.YES_NO_OPTION
);

if (confirm == javax.swing.JOptionPane.YES_OPTION) {
    if (kategoriDao.delete(kode)) {
        javax.swing.JOptionPane.showMessageDialog(this, "Kategori berhasil dihapus.");
        loadKategoriToTable(); // Refresh tabel
    } else {
        javax.swing.JOptionPane.showMessageDialog(this, "Gagal menghapus kategori.");
    }
}
}
}

```

Tampilan error:



C. Relasi Database (Foreign Key)

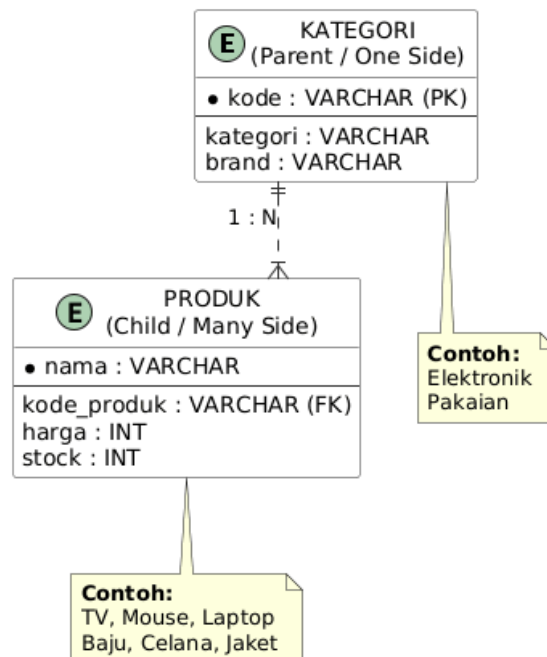
1. Struktur Database:

```
-- Tabel KATEGORI (Parent Table)
CREATE TABLE kategori (
    kode INT PRIMARY KEY AUTO_INCREMENT,
    kategori VARCHAR(100) NOT NULL,
    brand VARCHAR(100) NOT NULL
);

-- Tabel PRODUK (Child Table) dengan FOREIGN KEY
CREATE TABLE produk (
    kode INT PRIMARY KEY AUTO_INCREMENT,
    kode_produk INT, -- Foreign Key ke kategori.kode
    nama VARCHAR(100) NOT NULL UNIQUE,
    harga INT NOT NULL,
    stock INT NOT NULL,

    -- FOREIGN KEY CONSTRAINT
    CONSTRAINT produk_ibfk_1
        FOREIGN KEY (kode_produk)
        REFERENCES kategori (kode)
);
```

2. Visualisasi Relasi:



3. Query JOIN untuk Mengambil Data Lengkap:

```
public List<Produk> getAllProduk() {  
    List<Produk> list = new ArrayList<>();  
    String query = "SELECT p.kode, p.nama, p.harga, p.stock, p.kode_produk, k.kategori, k.brand " +  
        "FROM produk p INNER JOIN kategori k ON p.kode_produk = k.kode " +  
        "ORDER BY p.kode DESC";  
}
```

Penjelasan Query:

- `INNER JOIN` → Menggabungkan data dari kedua tabel
- `ON p.kode\_produk = k.kode` → Kondisi join (foreign key = primary key)
- Hasil: Data produk LENGKAP dengan nama kategori dan brand

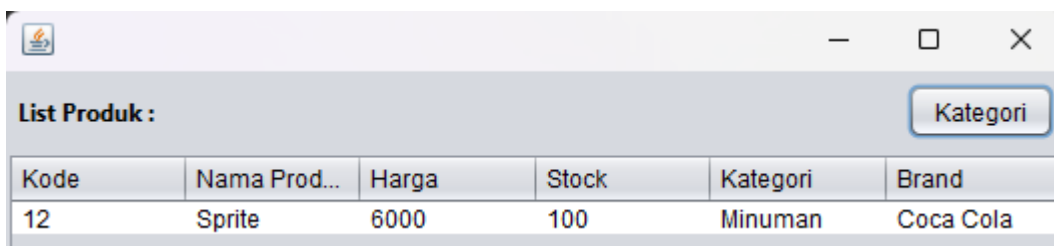
D. Fitur Kategori Terintegrasi

Pengertian: Fitur di mana Form Produk dan Form Kategori saling terhubung. Perubahan di Form Kategori akan mempengaruhi data yang ditampilkan di Form Produk.

1. Tombol Kategori di FormProduk:

```
private void btnkategoriActionPerformed(java.awt.event.ActionEvent evt) {  
    // Buat instance FormKategori  
    GUI.FormKategori formKat = new GUI.FormKategori();  
  
    // Tampilkan form kategori  
    formKat.setVisible(true);  
  
    // FormKategori bisa dimodifikasi, lalu ditutup  
    // Saat kembali ke FormProduk, user bisa klik Load untuk refresh data  
}
```

Tampilan:

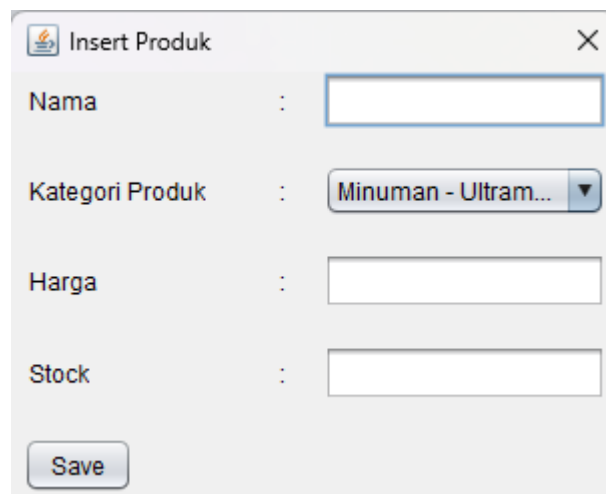


Kode	Nama Prod...	Harga	Stock	Kategori	Brand
12	Sprite	6000	100	Minuman	Coca Cola

2. Load Kategori ke ComboBox di DialogInputProduk:

```
private void loadKategoriData() {  
    Dao.KategoriDAO katDao = new Dao.KategoriDAO();  
  
    // Ambil semua data kategori dari database  
    listKategori = katDao.getAllKategori();  
  
    // Hapus isi default combobox (yang dibuat NetBeans)  
    cmbkategoriproduk.removeAllItems();  
  
    // Masukkan data kategori satu per satu ke combobox  
    for (Kategori kat : listKategori) {  
        cmbkategoriproduk.addItem(kat.getKategori() + " - " + kat.getBrand());  
    }  
}
```

Tampilan:



The screenshot shows a Java Swing dialog box titled "Insert Produk". It has a standard window with a title bar, a close button (X), and a maximize button. The dialog contains four labeled text fields: "Nama", "Kategori Produk", "Harga", and "Stock". The "Kategori Produk" field is a JComboBox showing the selected item "Minuman - Ultram...". At the bottom left, there is a "Save" button.

3. Penggunaan Kategori di DialogInputProduk:

```

private void btnSaveActionPerformed(java.awt.event.ActionEvent evt) {
    // Ambil semua input dari form
    String nama = txtnama.getText().trim();
    int kategoriIndex = cmbkategoriproduk.getSelectedIndex();
    String hargaStr = txtharga.getText().trim();
    String stokStr = txtstok.getText().trim();

    // VALIDASI 1: Cek apakah ada data yang kosong
    if (nama.isEmpty() || kategoriIndex == -1 || hargaStr.isEmpty() || stokStr.isEmpty()) {
        javax.swing.JOptionPane.showMessageDialog(this, "Semua data harus diisi!");
        return;
    }

    try {
        // VALIDASI 2: Konversi harga dan stok ke angka
        int harga = Integer.parseInt(hargaStr);
        int stok = Integer.parseInt(stokStr);

        // VALIDASI 3: Cek apakah nilai negatif
        if (harga < 0 || stok < 0) {
            javax.swing.JOptionPane.showMessageDialog(this,
                "Harga dan Stok tidak boleh negatif!");
            return;
        }

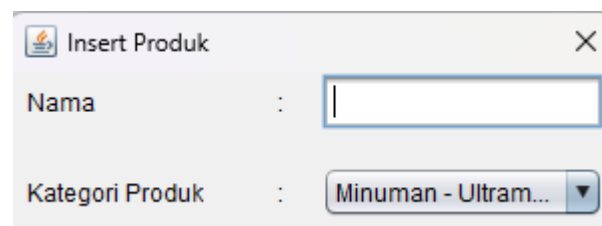
        // Ambil kategori yang dipilih dari combobox
        Kategori katTerpilih = listKategori.get(kategoriIndex);

        // Bungkus data ke objek Produk
        Produk p = new Produk();
        p.setNama(nama);
        p.setKodeProduk(katTerpilih.getKode());
        p.setHarga(harga);
        p.setStok(stok);

        // Simpan ke database
        if (produkDao.insert(p)) {

```

Tampilan:



The screenshot shows a window titled "Insert Produk" with a close button (X) in the top right corner. Inside the window, there are two labels with corresponding input controls:

- Nama :** followed by a text input field.
- Kategori Produk :** followed by a dropdown menu currently displaying "Minuman - Ultram..." with a downward arrow on the right.

E. Fitur Update dengan Dialog Internal

Pengertian: Fitur yang memungkinkan user mengedit data produk melalui dialog terpisah yang muncul saat tombol Update diklik.

1. Tombol Update di FormProduk:

```

private void btnupdateActionPerformed(java.awt.event.ActionEvent evt) {
    int selectedRow = tablelistproduk.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Pilih produk yang akan diupdate.");
        return;
    }

    int kodeProduk = (int) tablelistproduk.getValueAt(selectedRow, 0);

    Produk p = produkDao.getProdukById(kodeProduk);
    if (p == null) {
        JOptionPane.showMessageDialog(this, "Data produk tidak ditemukan.");
        return;
    }

    showUpdateDialog(p);
}

```

2. Method showUpdateDialog:

```

private void showUpdateDialog(Produk produkLama) {
    // Buat JDialog internal
    JDialog updateDialog = new JDialog(this, "Update Produk", true);

    // Ambil data kategori dari database
    Dao.KategoriDAO katDao = new Dao.KategoriDAO();
    java.util.List<Kategori> listKategori = katDao.getAllKategori();

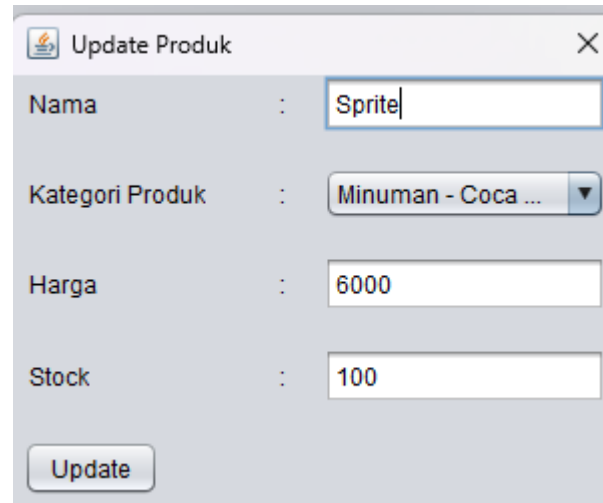
    // === Buat komponen-komponen form ===
    JLabel jLabel1 = new JLabel("Nama");
    JTextField txtnama = new JTextField();
    txtnama.setText(produkLama.getNama());

    JLabel jLabel2 = new JLabel("Kategori Produk");
    JComboBox<String> cmbkategoriproduk = new JComboBox<>();
    for (Kategori kat : listKategori) {
        cmbkategoriproduk.addItem(kat.getKategori() + " - " + kat.getBrand());
    }

    // Set selected index sesuai produk lama
    for (int i = 0; i < listKategori.size(); i++) {
        if (listKategori.get(i).getKode() == produkLama.getKodeProduk()) {
            cmbkategoriproduk.setSelectedIndex(i);
            break;
        }
    }
}

```

Tampilan:



The image shows a software window titled "Update Produk" with a close button (X) in the top right corner. Inside the window, there are four labeled input fields arranged vertically: "Nama" (Name) containing the text "Sprite", "Kategori Produk" (Product Category) with a dropdown menu currently showing "Minuman - Coca ...", "Harga" (Price) containing the number "6000", and "Stock" (Inventory) containing the number "100". At the bottom left of the window is a button labeled "Update".

7. KESIMPULAN

Aplikasi Manajemen Toko ini telah menerapkan berbagai konsep dan praktik pemrograman yang baik:

A. Ringkasan Fitur

1. Manajemen Produk

- Tambah produk baru dengan dialog terpisah
- Edit produk yang ada
- Hapus produk
- Lihat semua produk dalam tabel

2. Manajemen Kategori

- Tambah kategori baru
- Edit kategori
- Hapus kategori (dengan validasi relasi)
- Lihat semua kategori

3. Integritas Data

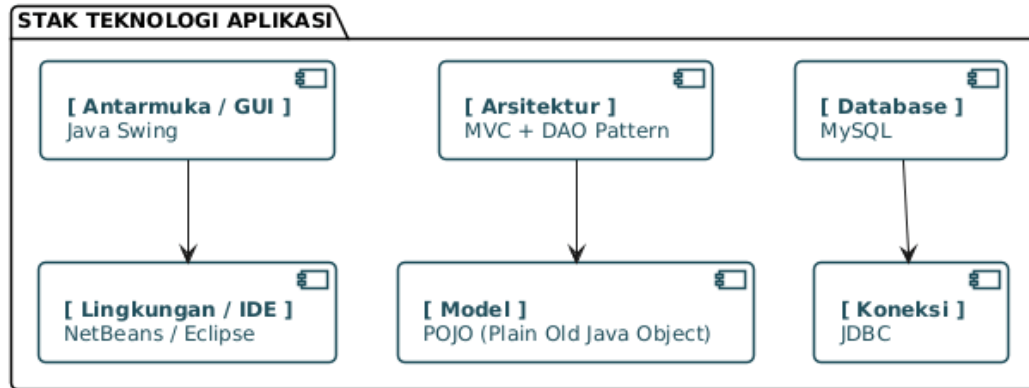
- Validasi input (kosong, tipe data, duplikat)
- Cek relasi foreign key sebelum hapus
- Konfirmasi sebelum hapus data

4. User Experience

- Auto refresh setelah CRUD

- Form terintegrasi (Produk ↔ Kategori)
- Dialog modal untuk input/update
- Pesan error yang informatif

B. Teknologi yang Digunakan



PENUTUP

Aplikasi Manajemen Toko ini adalah contoh implementasi aplikasi desktop Java yang menerapkan konsep OOP, pola desain, dan praktik pemrograman yang baik. Dengan fitur CRUD yang lengkap, validasi yang ketat, dan integritas data yang terjaga, aplikasi ini siap digunakan untuk mengelola data produk dan kategori secara efisien.

Semoga laporan ini bermanfaat untuk memahami alur dan implementasi aplikasi.